PROPOSED MODEL / SYSTEM DESIGN

(Flowcharts / Block Diagrams / Algorithms / DFD / ER Diagrams)

and Implementation of Code

Project Title

HireSense AI

An AI-Based Resume Screening and Recruitment Assistant

Submitted in partial fulfilment of the requirements for the degree of

[Bachelor of Technology in Computer Science and Engineering]

Submitted by

[Student Name] – [Enrollment / Roll No.]

[Student Name] – [Enrollment / Roll No.]

Under the guidance of

[Guide Name]

[Designation, Department]

[Department of Computer Science and Engineering]

[Name of College / University]

Academic Year 2026 – 2027

TABLE OF CONTENTS

1. Introduction 3
2. Proposed Model 3
3. System Architecture (Block Diagram) 4
4. Flowcharts 6
5. Algorithms 7
6. Data Flow Diagrams (DFD) 9
7. Entity-Relationship (ER) Diagram and Database Design 12
8. Implementation of Code 14
9. User Interface and Running Model 17
10. Testing 19
11. Conclusion 20
12. References 21

List of figures: Block diagram (Fig. 3.1), Recruiter flowchart (Fig. 4.1), Candidate flowchart (Fig. 4.2), DFD level 0 (Fig. 6.1), DFD level 1 (Fig. 6.2), ER diagram (Fig. 7.1).

1. Introduction

This document presents the proposed model and the system design of the project “HireSense AI – An AI-Based Resume Screening and Recruitment Assistant”, together with the implementation of the code that realises the design. It follows the project synopsis and describes how the system is organised, how data flows through it, how the main algorithms work and how the running application is built and operated.

HireSense AI is a role-based web application for three kinds of users: candidates, recruiters and administrators. It extracts structured information from resumes, matches candidates with job descriptions using a language model, shows explainable scores, supports interview scheduling and gives candidates feedback to improve their resumes. The human recruiter always makes the final decision.

1.1 Scope of this Document

- Proposed model and overall design approach.
- System architecture as a block diagram.
- Flowcharts of the recruiter and candidate processes.
- Algorithms in pseudocode.
- Data Flow Diagrams (level 0 and level 1).
- ER diagram and database design.
- Implementation details: technology, project structure, key code, API endpoints and security.
- User interface, instructions to run the model and test cases.

2. Proposed Model

The proposed model combines a conventional three-tier web application with a language-model service. A resume is treated as unstructured text; the model converts it into structured data and a match assessment, and a verification step makes sure that nothing in the result is invented. The recruiter or candidate then works with explainable results instead of a bare score.

2.1 Model Overview

1. **Input:** resumes in PDF or text format and job descriptions with required and optional skills.
2. **Text extraction:** the file is converted to plain text.

3. **AI analysis:** the language model returns a fixed JSON structure with the candidate profile, per-skill match, sub-scores (skills, experience, domain, education), reasons and a recommendation.
4. **Verification:** each extracted item is checked against the resume text; unsupported items are removed (hallucination control).
5. **Decision support:** scores, matched and missing skills and reasons are shown; the recruiter shortlists, reviews or rejects.
6. **Feedback loop:** recruiter decisions are compared with AI recommendations and used to suggest a better shortlisting threshold.

2.2 Users and Their Functions

User	Main functions
Candidate	Register/login, manage profile, upload resume versions, ATS report, improvement suggestions, skill gap analysis, version comparison, job recommendations, apply to jobs, track applications, generative career assistant.
Recruiter	Company profile, create and publish jobs with required/optional skills, batch resume screening, ranked and explainable candidate reports, candidate comparison, shortlist/reject, interview questions, interview scheduling, HR chatbot, analytics, blind screening and fairness audit.
Administrator	Manage users and job postings, platform statistics, skills demand analytics, shortlisting statistics, API usage and error monitoring, audit logs, shortlist threshold and feedback learning.

Table 2.1: Users and functions

2.3 Design Principles

- **Explainability:** every score is accompanied by reasons and skill-level evidence.
- **Verification before trust:** AI output is validated against the source document.
- **Human in the loop:** AI recommends, a person decides.
- **Least privilege:** each API route checks the role of the logged-in user.
- **Simplicity:** one Next.js project contains both frontend and backend, and one MySQL database stores all data.

3. System Architecture (Block Diagram)

The system is organised in four layers. The browser-based presentation layer talks to server-side API routes in the application layer. The application layer reads and writes the MySQL database, extracts text with the pdf-parse library and calls the AI service through one wrapper function that logs usage and errors.

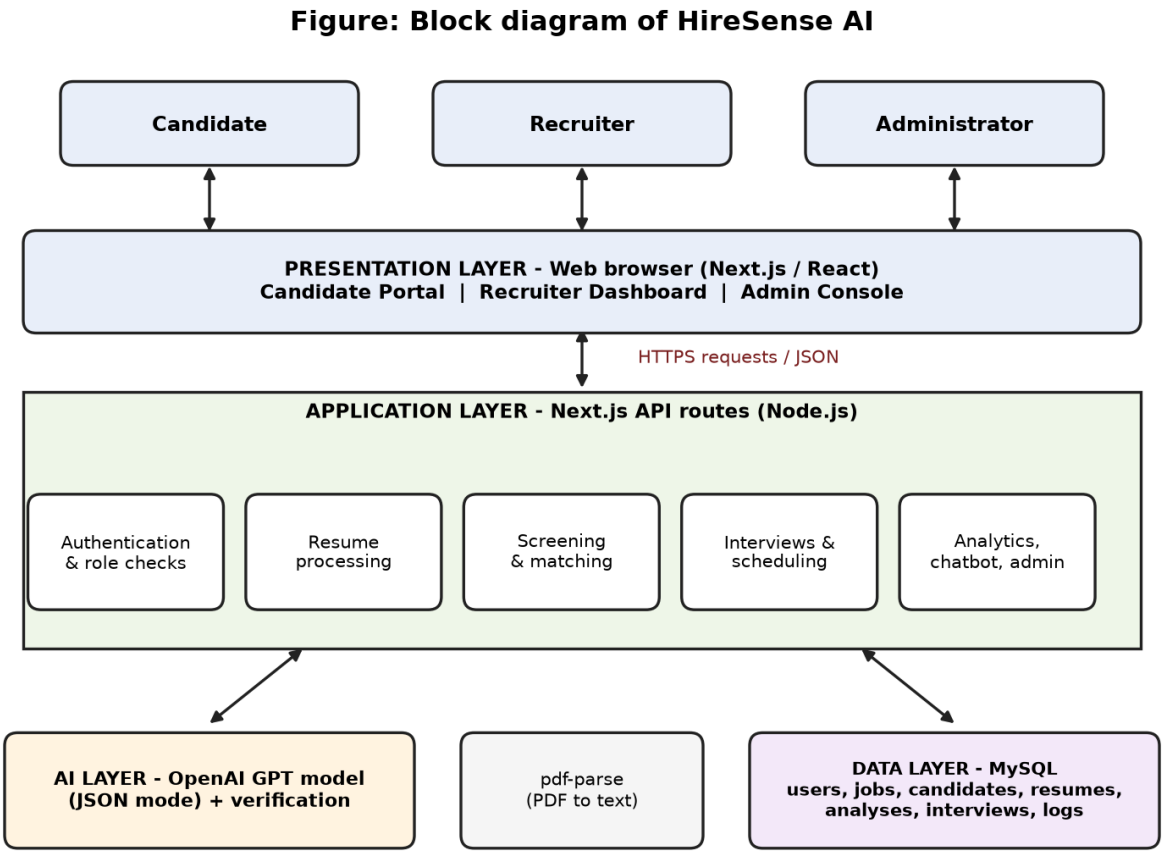


Figure 3.1: Block diagram of HireSense AI

Layer	Components	Responsibility
Presentation	Next.js / React pages: candidate portal, recruiter dashboard, admin console; SVG charts	User interface, form validation, calls to the API.
Application	Next.js API routes on Node.js: authentication and role checks, resume processing, screening and matching, interviews, analytics, chatbot, administration	Business logic, access control, orchestration of AI and database.
AI	OpenAI GPT model in JSON mode, verification functions	Parsing, semantic matching, explanations, ATS and skill gap analysis, question generation, career assistant.
Data	MySQL 8 with JSON columns	Persistent storage of users, jobs, resumes, candidates, analyses, interviews, feedback and logs.

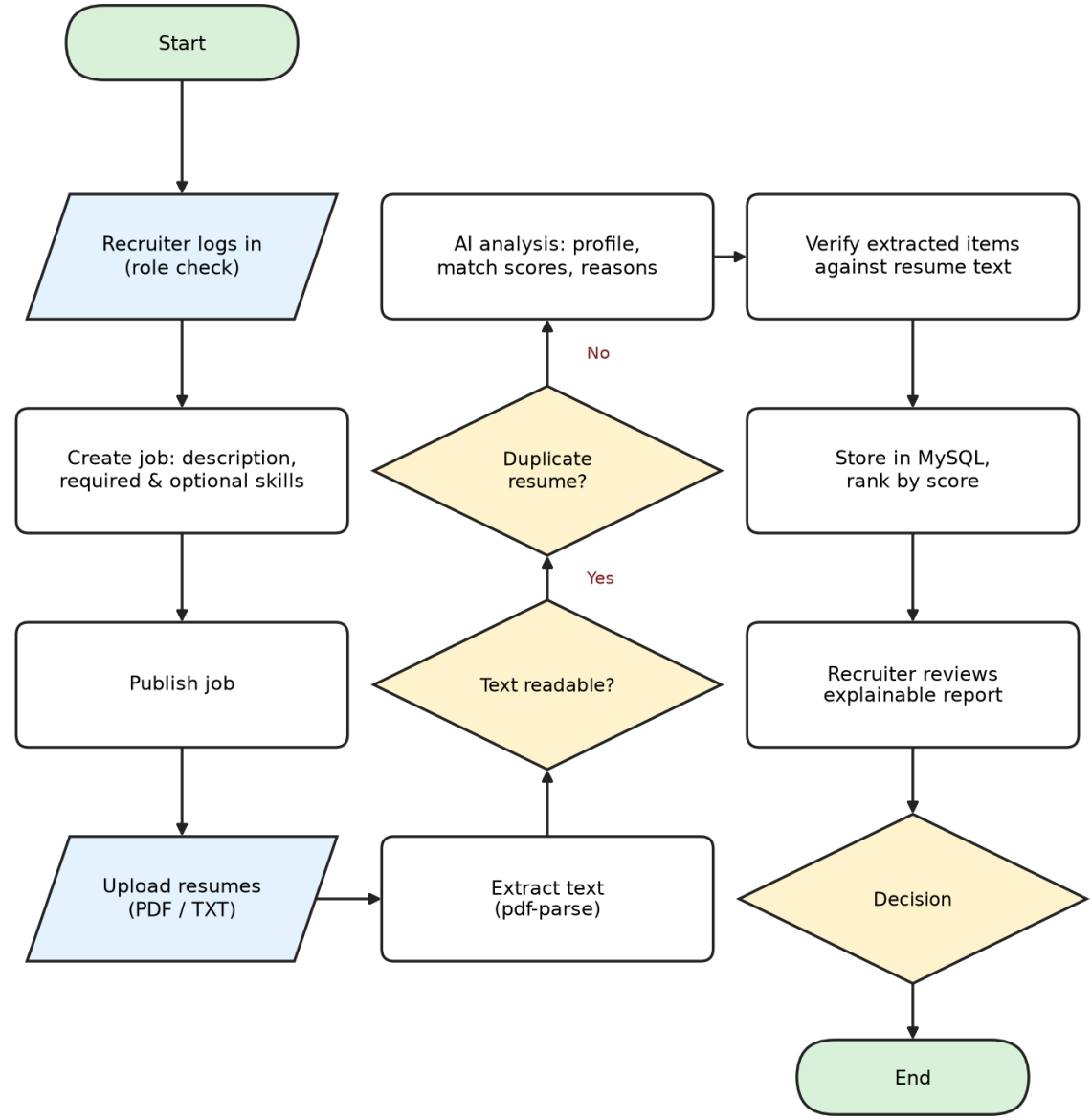
Table 3.1: Layers of the architecture

4. Flowcharts

4.1 Recruiter Flow

Flowchart 1 shows how a recruiter creates a job and screens resumes. Unreadable files and duplicate resumes are skipped with a message. Accepted resumes are analysed by the AI, verified against the text, stored and ranked; the recruiter then reviews the explainable report and decides.

Flowchart 1: Recruiter - job creation and resume screening



Text readable? - No: friendly error is shown and the file is skipped. Duplicate resume? - Yes: file is skipped.
Decision: Shortlist / Keep for review / Reject; shortlisted candidates move on to interview scheduling.

Figure 4.1: Flowchart of recruiter job creation and resume screening

4.2 Candidate Flow

Flowchart 2 shows the candidate journey: upload and validate a resume, receive the structured profile and improvement reports, compare versions, apply to a recommended job and track the result.

Flowchart 2: Candidate - resume analysis and job application

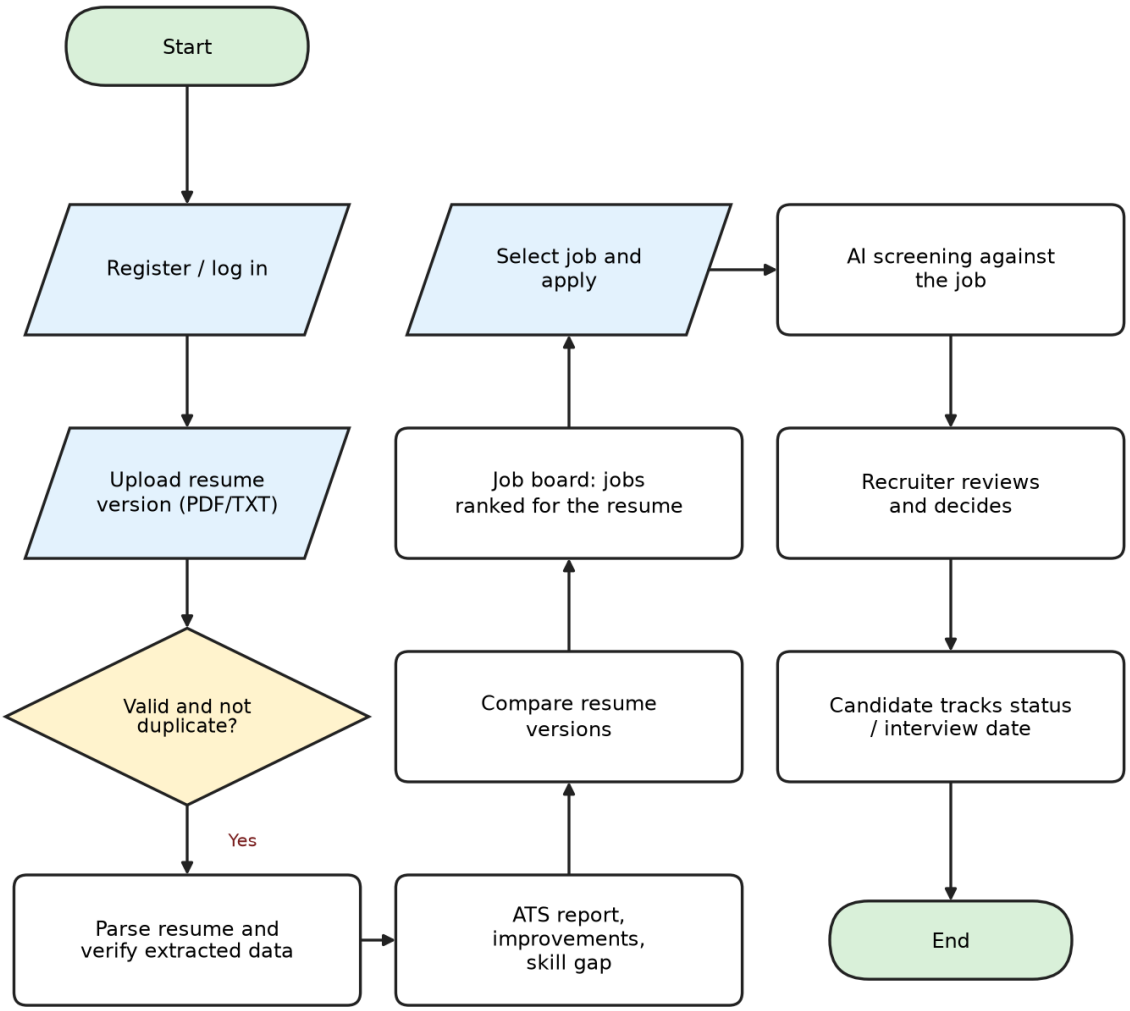


Figure 4.2: Flowchart of candidate resume analysis and job application

5. Algorithms

The main algorithms of the system are given below in pseudocode. They correspond to functions in the implemented code.

Algorithm 1: Batch resume screening

```
Input : job J (required skills R, optional skills O), resume files F1..Fn
Output: scored and ranked candidate records stored in the database
1.  threshold <- shortlist threshold from settings (default 80)
2.  seen <- empty set of resume hashes
3.  for each file f in F (in parallel):
4.      text <- extract_text(f)              // pdf-parse or plain text
5.      if text is empty or file type unsupported: report error; continue
6.      h <- SHA256(normalise(text))
7.      if h already stored for job J or h in seen: report duplicate; continue
8.      add h to seen
9.      result <- LLM(J, text)               // JSON: scores, overlap, reasons, profile
10.     result.profile <- VERIFY(result.profile, text)      // Algorithm 2
11.     if score >= threshold          then status <- Shortlisted
        else if score >= threshold-20  then status <- Review
        else                                status <- Rejected
12.     store candidate (scores, profile, hash, status)
13. rank the candidates of J by score in descending order
```

Algorithm 2: Hallucination control (verification of extracted data)

```
Input : extracted profile P, resume text T
Output: verified profile P', counts (checked, removed)
1. H <- lower-case(T) with whitespace collapsed
2. checked <- 0; removed <- 0
3. for each item x in education, experience, projects, certifications, skills:
4.     checked <- checked + 1
5.     if no key field of x (institution/degree, company/role, name, text)
           occurs as a substring of H:
6.         remove x from P; removed <- removed + 1
7. keep e-mail only if it occurs in T; keep phone only if its digits occur in digits(T)
8. return P', (checked, removed)
```

Algorithm 3: Job recommendation for a candidate

```
Input : resume text T, published jobs
1. for each job j:
2.     R <- required skills of j; O <- optional skills of j
3.     foundR <- number of skills in R that occur in T (case-insensitive)
4.     foundO <- number of skills in O that occur in T
5.     total  <- 2*|R| + |O|
6.     match(j) <- round(100 * (2*foundR + foundO) / total)   if total > 0
7. sort jobs by match(j) in descending order
```

Algorithm 4: Interview slot conflict check

```
Input : start time S, duration d (minutes), candidate c
1. end <- S + d
2. if there is an interview I with I.start < end AND I.start + I.duration > S:
3.     reject the booking (overlap)
4. else store the interview and draft the invitation e-mail
```

Algorithm 5: Feedback learning (shortlist threshold)

```
1. whenever a recruiter changes a candidate status, store
      (AI recommendation, final status, score)
2. agreement <- (number of decisions equal to the AI recommendation) / total
3. H <- scores of candidates finally shortlisted by recruiters
4. if |H| >= 3:  suggested <- max(50, floor(min(H) / 5) * 5)
5. the administrator may apply the suggestion (or any value from 50 to 95)
      as the new shortlist threshold used in Algorithm 1
```

Algorithm 6: Fairness (selection-rate) audit

```
1. group candidates by experience band and by city (groups with >= 2 candidates)
2. rate(g)  <- shortlisted(g) / total(g)
3. best     <- maximum rate over all groups
4. ratio(g) <- rate(g) / best
5. flag group g when ratio(g) < 0.80       // four-fifths rule
```

Algorithm 7: Resume version comparison

```
Input : resume versions A and B of one candidate
1. get the ATS score and the technical skills of A and B
2. delta   <- ATS(B) - ATS(A)
3. added   <- skills(B) minus skills(A);  removed <- skills(A) minus skills(B)
4. report delta, added, removed, word counts and number of sections
```

6. Data Flow Diagrams (DFD)

6.1 DFD Level 0 (Context Diagram)

At level 0 the whole system is one process that exchanges data with three kinds of users and with the external AI service.

Figure: DFD Level 0 (context diagram)

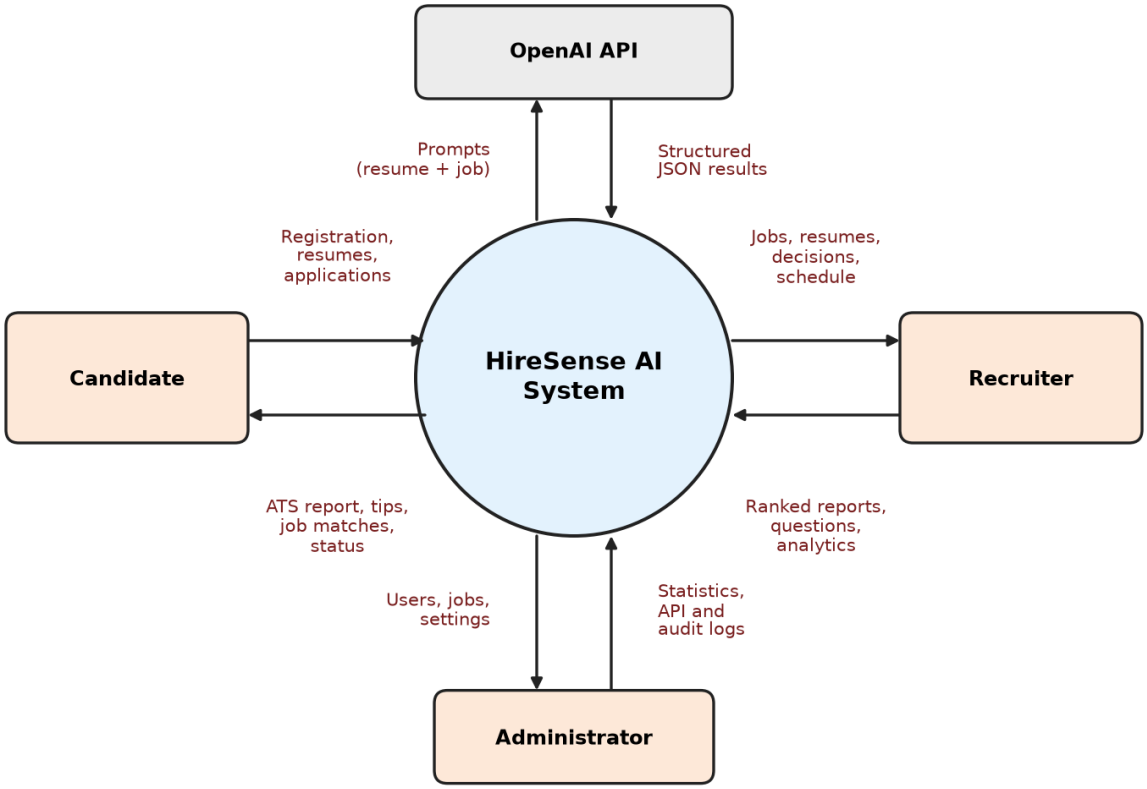


Figure 6.1: DFD level 0

6.2 DFD Level 1

At level 1 the system is divided into six processes that use five data stores. All users log in through process 1.0.

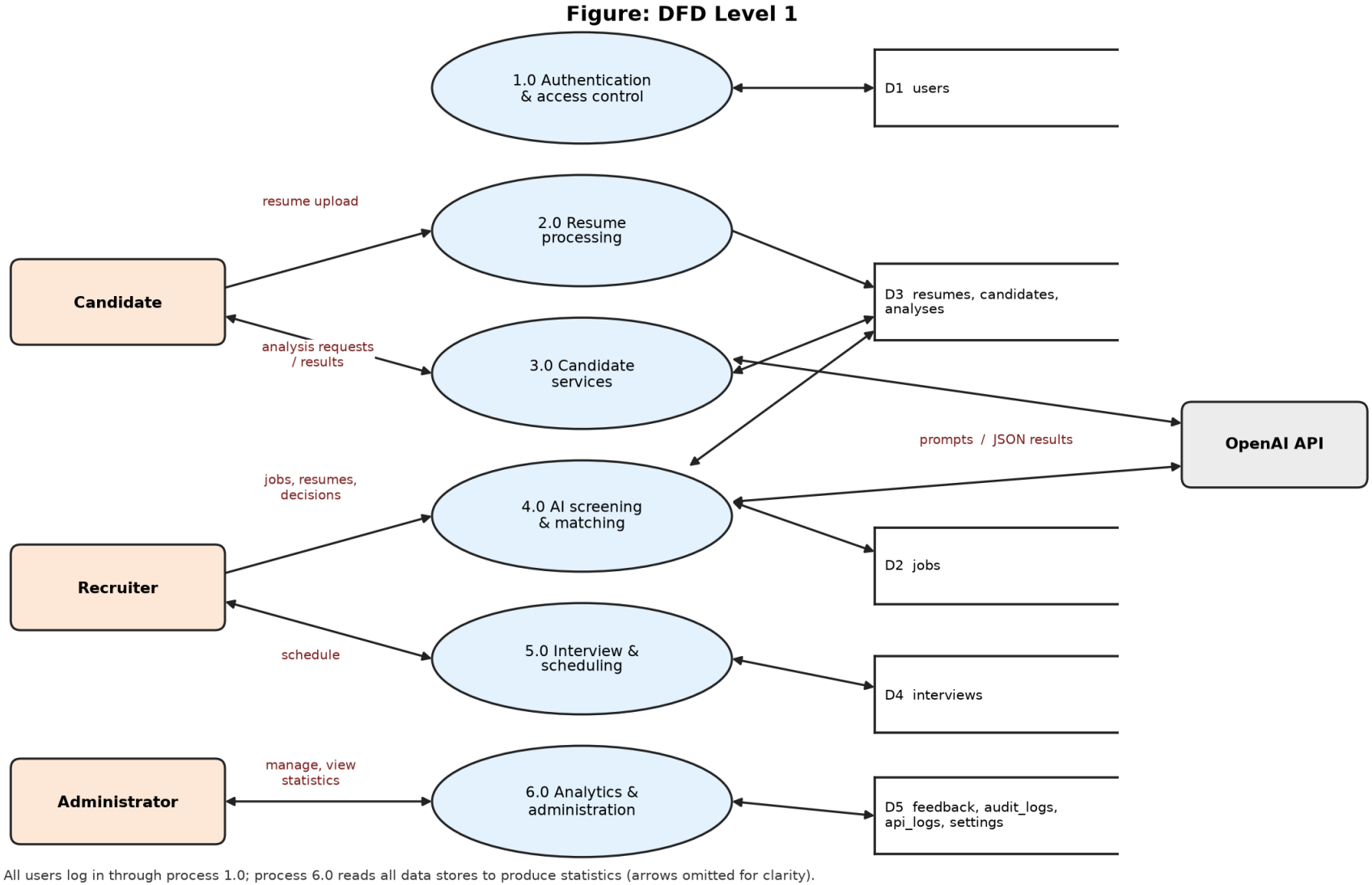
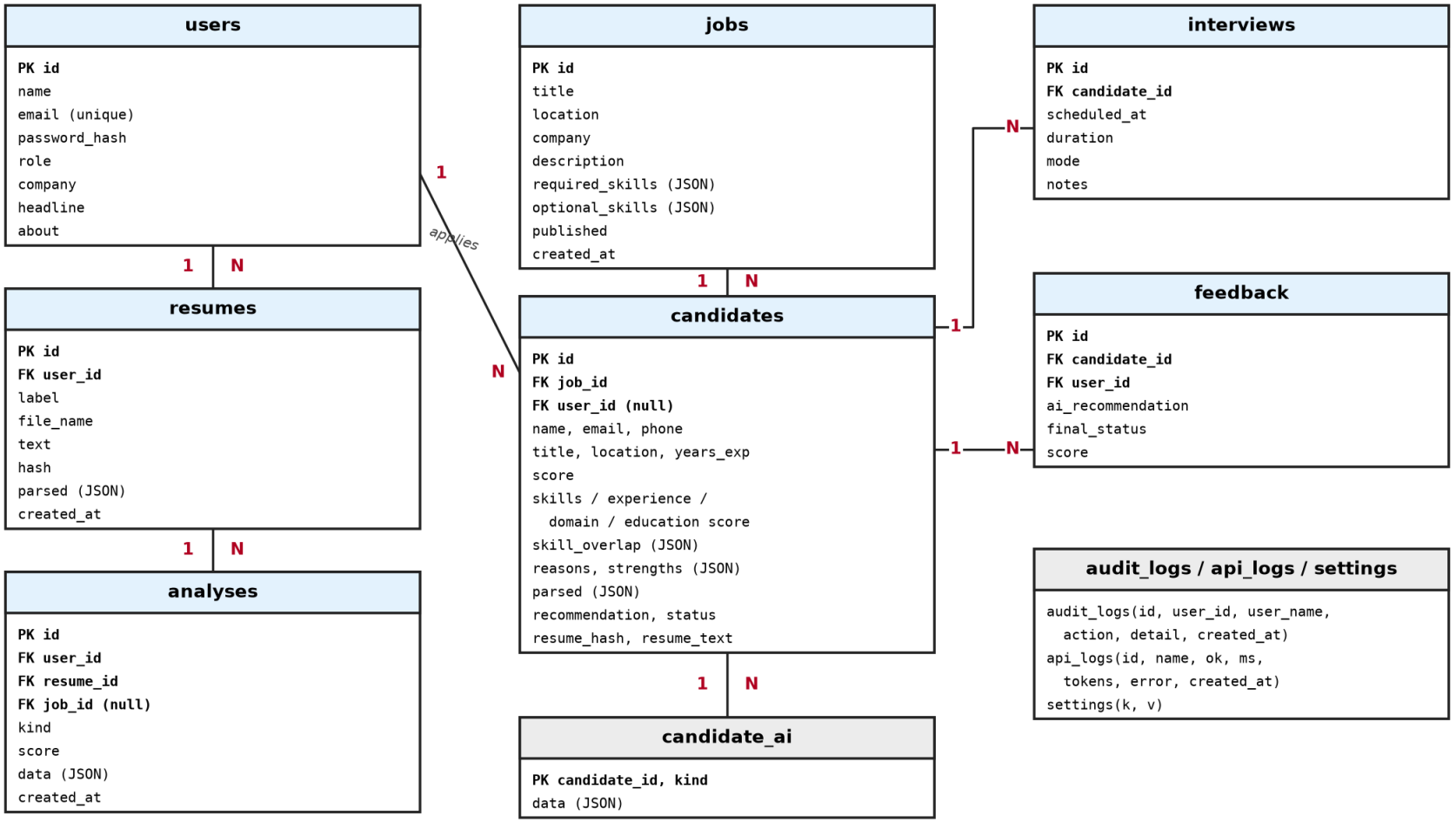


Figure 6.2: DFD level 1

7. Entity-Relationship (ER) Diagram and Database Design

Figure: Entity-Relationship (ER) diagram



Legend: PK = primary key, FK = foreign key, 1 / N = one-to-many.

candidates.user_id marks an application by a candidate account; analyses.job_id is an optional link to jobs; feedback.user_id and audit_logs.user_id refer to users.

Figure 7.1: ER diagram of the HireSense AI database

6.3 Description of Processes and Data Stores

No.	Process	Description
1.0	Authentication and access control	Registers users, verifies passwords, creates the session and checks the role on every request.
2.0	Resume processing	Accepts uploads, extracts text, detects duplicates and stores resume text and structured profiles.
3.0	Candidate services	ATS report, improvement suggestions, skill gap analysis, version comparison, job board, applications and career assistant.
4.0	AI screening and matching	Compares resumes with jobs, produces scores, reasons and recommendations, verifies the output and ranks candidates.
5.0	Interview and scheduling	Generates interview questions, books slots without overlap and drafts invitations.
6.0	Analytics and administration	Statistics, skills demand, fairness audit, user and job management, API monitoring and audit logs.

Table 6.1: Processes of DFD level 1

Store	Contents
D1 users	Accounts, roles, company and profile details.
D2 jobs	Job descriptions, required and optional skills, publication status.
D3 resumes, candidates, analyses	Resume versions, screened candidates with scores and structured profiles, analysis history.
D4 interviews	Scheduled interviews.
D5 feedback, audit_logs, api_logs, settings	Recruiter feedback, audit trail, AI usage and errors, shortlist threshold.

Table 6.2: Data stores

7.1 Relationships

Relationship	Cardinality	Meaning
users – resumes	1 : N	A candidate account owns many resume versions.
resumes – analyses	1 : N	Each resume has many saved analyses (ATS, improvement, skill gap).
jobs – candidates	1 : N	A job has many screened or applied candidates.
users – candidates	1 : N (optional)	A candidate account can apply to many jobs; recruiter-screened resumes have no user.
candidates – interviews	1 : N	A candidate can have scheduled interviews.
candidates – feedback	1 : N	Every recruiter decision is stored as feedback.
candidates – candidate_ai	1 : N	Cached AI outputs (skill gap, ATS, questions) per candidate.

Table 7.1: Relationships between entities

7.2 Main Tables

Table	Key columns	Notes
users	id, name, email, password_hash, role, company, headline, about	Roles: admin, recruiter, candidate. Passwords stored as scrypt hashes.
jobs	id, title, location, company, description, required_skills, optional_skills, published	Skill lists are JSON arrays.
candidates	id, job_id, user_id, name, email, score, skills/experience/domain/education score, skill_overlap, reasons, strengths, parsed, recommendation, status, resume_hash	status is Shortlisted, Review or Rejected; resume_hash supports duplicate detection.
resumes	id, user_id, label, file_name, text, hash, parsed	Multiple versions per candidate.
analyses	id, user_id, resume_id, job_id, kind, score, data	Analysis history; kind is ats, improve or gap.
interviews	id, candidate_id, scheduled_at, duration, mode, notes	Overlap check before insert.
feedback	id, candidate_id, user_id, ai_recommendation, final_status, score	Used for feedback learning.
audit_logs, api_logs, settings	action/detail; name/ok/ms/tokens/error; k/v	Monitoring, audit trail and shortlist threshold.

Table 7.2: Main database tables

8. Implementation of Code

8.1 Development Environment

Item	Details
Language / runtime	JavaScript (ES2020+), Node.js 18 or later
Framework	Next.js 14 (App Router) with React 18; frontend and backend in one project
Database	MySQL 8 using the mysql2 driver; database name hireai, tables created automatically
AI service	OpenAI Chat Completions API, model gpt-4o-mini, JSON response format
Libraries	pdf-parse (PDF text extraction), openai, mysql2, next, react, react-dom
Tools	VS Code, Git/GitHub, MySQL Workbench / command-line client, web browser

Table 8.1: Development environment

8.2 Project Structure

```
AI-Based Recruitment/
├── app/
│   ├── page.js        app shell, dashboard, candidate list, match details
```

```
│   ├── portal.js      candidate portal (resumes, job board, applications, career assistant)
│   ├── admin.js       recruiter jobs page and admin console
│   ├── features.js    skill gap, ATS view, questions, compare, scheduling, chatbot,
analytics
│   ├── ui.js          fetch helpers and SVG charts
│   ├── layout.js, globals.css
│   └── api/           login, register, profile, jobs, screen, candidates, ai, chat,
│                      interviews, report, blind, resumes, resume-ai, career, board,
│                      applications, admin   (one route.js each)
├── lib/
│   ├── server.js      database, auth, role checks, AI wrapper, parsing, verification
│   ├── prompts.js     AI prompts
│   └── sample.js      sample data
├── database/sample-data.sql
├── package.json, .env.example, README.md
```

8.3 Key Parts of the Code

The following excerpts (shortened) show the core logic. They are taken from lib/server.js and from the API routes.

Code 8.1: Hallucination control (lib/server.js)

```
export function verifyProfile(p, text) {
  const hay = text.toLowerCase().replace(/\s+/g, " ");
  const has = (s) => !!s && hay.includes(String(s).toLowerCase().trim());
  let checked = 0, removed = 0;
  const keep = (arr, test) => (Array.isArray(arr) ? arr : []).filter((x) => {
    checked++; const ok = test(x); if (!ok) removed++; return ok;
  });
  return {
    profile: {
      education: keep(p.education, (e) => has(e?.institution) || has(e?.degree)),
      experience: keep(p.experience, (e) => has(e?.company) || has(e?.role)),
      technicalSkills: keep(p.technicalSkills, has),
      /* projects, certifications, softSkills handled the same way */
    },
    verification: { checked, removed },
  };
}
```

Code 8.2: Duplicate detection and screening (app/api/screen/route.js)

```
const h = hashText(text);
const [dup] = await pool.query(
  "SELECT name FROM candidates WHERE job_id = ? AND resume_hash = ?", [jobId, h]);
if (dup.length || seen.has(h)) throw new Error("Duplicate resume - skipped");
seen.add(h);
const r = await analyze(job, text);
await insertCandidate(pool, { jobId, fileName, text, r,
                              status: statusFor(r.score, threshold) });
```

Code 8.3: Status from threshold and role check (lib/server.js)

```
export const statusFor = (score, th) =>
  score >= th ? "Shortlisted" : score >= th - 20 ? "Review" : "Rejected";


export async function getStaff() {
  const u = await getUser();
  return u && (u.role === "admin" || u.role === "recruiter") ? u : null;
}
```

Code 8.4: Job recommendation score (app/api/board/route.js)

```
const total = req.length * 2 + opt.length;
const got = req.filter(has).length * 2 + opt.filter(has).length;
match = hay && total ? Math.round((got / total) * 100) : null;
```

Code 8.5: Interview overlap check (app/api/interviews/route.js)

```
const [clash] = await pool.query(
  `SELECT id FROM interviews
   WHERE scheduled_at < DATE_ADD(?, INTERVAL ? MINUTE)
     AND DATE_ADD(scheduled_at, INTERVAL duration MINUTE) > ?`,
  [start, mins, start]);
if (clash.length) return Response.json(
  { error: "That time overlaps another interview." }, { status: 409 });
```

Code 8.6: Single wrapper for AI calls with usage/error logging (lib/server.js)

```
export async function callOpenAI(name, params) {
  const t = Date.now();
  try {
    const client = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });
    const r = await client.chat.completions.create({ model: "gpt-4o-mini", ...params });
    /* log name, duration and tokens to api_logs */
    return r;
  } catch (e) {
    /* log the error to api_logs */
    throw e;
  }
}
```

8.4 API Endpoints

Endpoint	Methods	Access	Purpose
/api/login, /api/register	GET, POST, DELETE / POST	Public	Session check, login, logout; sign-up.
/api/profile	PUT	All roles	Update profile / company profile.
/api/jobs	GET, POST, PATCH	Staff	List, create, publish/unpublish jobs.
/api/screen	POST	Staff	Batch screening of resumes for a job.
/api/candidates	GET, PATCH	Staff	Filtered list, details, status change (feedback stored).
/api/ai	POST	Staff	Skill gap, ATS analysis, interview questions per candidate.
/api/chat	POST	Staff	HR chatbot answering from the candidate database.
/api/interviews	GET, POST, DELETE	Staff	Schedule, list and cancel interviews; invitation draft.

Endpoint	Methods	Access	Purpose
/api/report, /api/blind	GET / GET, POST	Staff	CSV report; blind screening switch.
/api/resumes	GET, POST, DELETE	Candidate	Resume versions.
/api/resume-ai	GET, POST	Candidate	ATS report, improvements, skill gap, history.
/api/career	POST	Candidate	Generative career assistant.
/api/board, /api/applications	GET, POST / GET	Candidate	Job recommendations, apply, application status.
/api/admin	GET, PATCH, DELETE	Admin	Statistics, users, jobs, API monitor, audit logs, threshold.

Table 8.2: API endpoints

8.5 Security Measures

- Passwords are hashed with scrypt and a random salt; the session is a signed HttpOnly cookie.
- Every API route verifies the role; candidates cannot call recruiter or admin routes.
- The OpenAI key and database password live only in the server-side .env.local file, which is excluded from Git.
- SQL statements use parameters (prepared values) to prevent SQL injection.
- Blind screening removes names and contact details on the server before data reaches the browser.
- Important actions are written to the audit log.

9. User Interface and Running Model

9.1 How to Run the Model

1. Install Node.js 18+ and MySQL 8 and start the MySQL server.
2. Copy .env.example to .env.local and set OPENAI_API_KEY and the MYSQL_* values.
3. Open a terminal in the project folder and run npm install.
4. Run npm run dev and open http://localhost:3000. The database, tables and sample data are created automatically on the first request.
5. Sign in with a demo account (below) or register a new candidate or recruiter account.

Role	Email	Password
Administrator	admin@hireai.com	admin123
Recruiter	recruiter@hireai.com	recruiter123

Role	Email	Password
Candidate	candidate@hireai.com	candidate123

Table 9.1: Demo accounts (for local use only; change before any public deployment)

9.2 Application Screens

The screens below show the running model. Paste the screenshots taken from the running application in the marked boxes.

[Paste screenshot here]

Figure 9.1: Login and registration screen

[Paste screenshot here]

Figure 9.2: Recruiter dashboard with best-match card and statistics

[Paste screenshot here]

Figure 9.3: Candidate match details with explainable scores

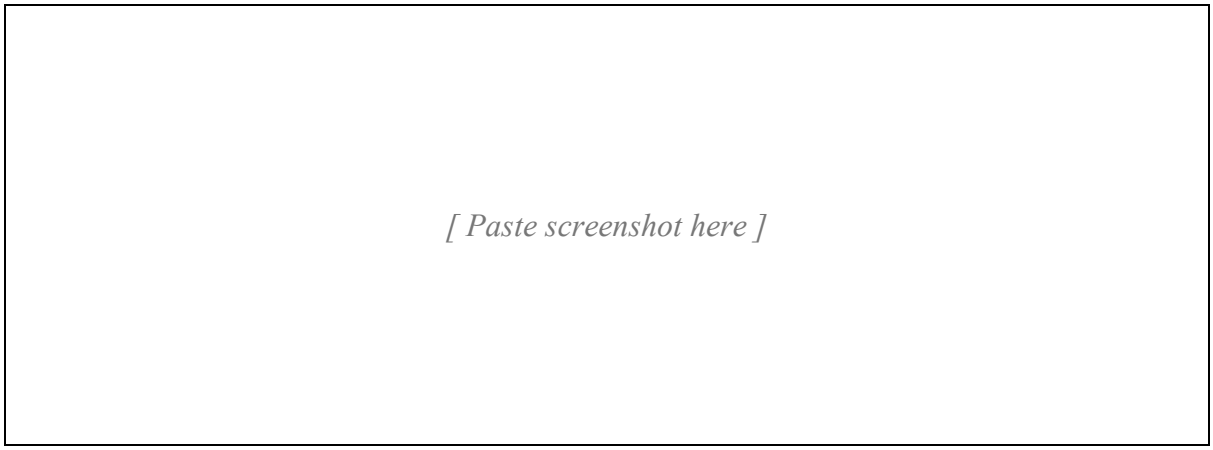


Figure 9.4: Skill gap analysis (radar chart) and ATS analysis

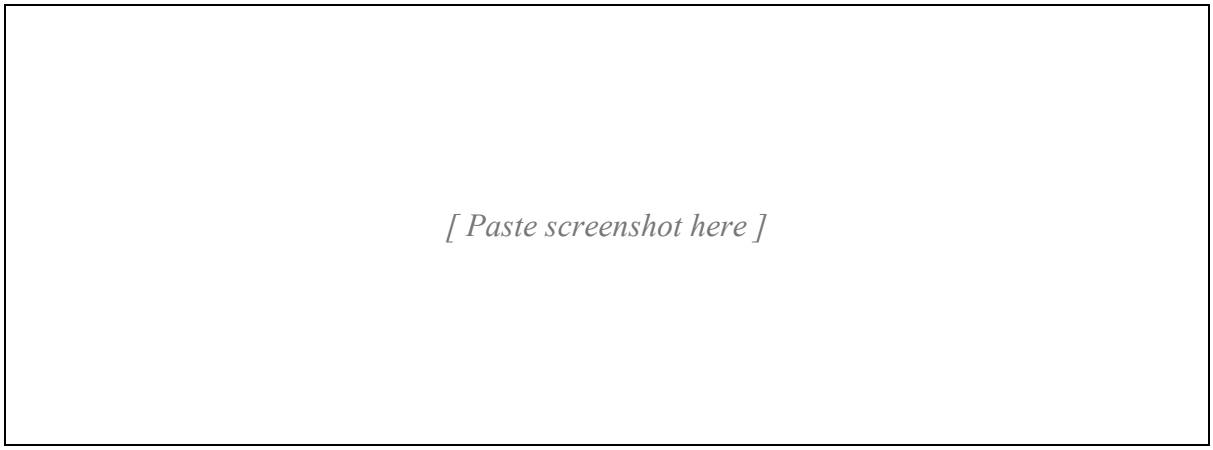


Figure 9.5: Candidate portal – resume versions and job board

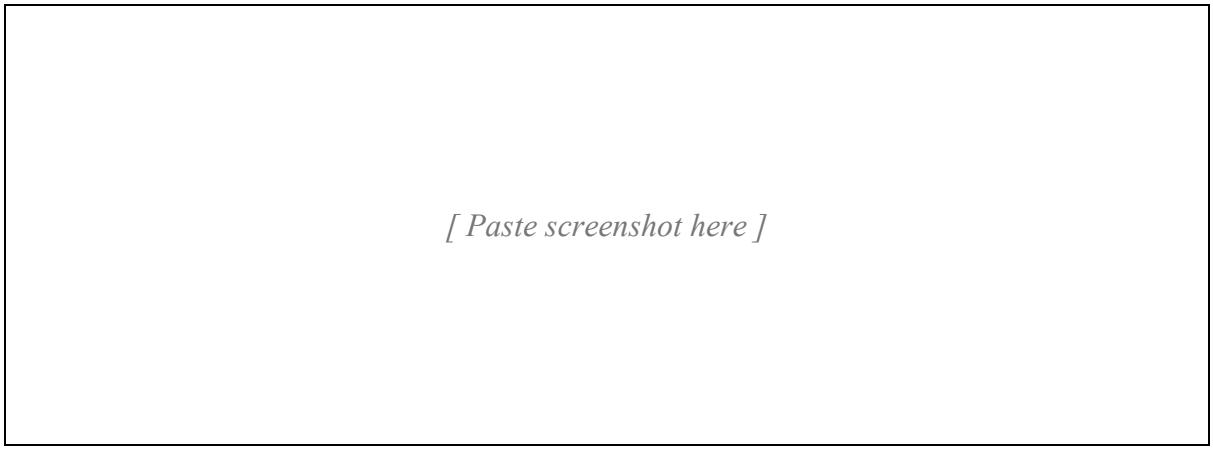


Figure 9.6: Admin console – API monitoring and audit logs

10. Testing

The following test cases are used to verify the running model. The Result column is to be completed with Pass or Fail after executing the tests on the running application.

ID	Test case	Input / action	Expected result	Result
TC1	Login with wrong password	Valid email, wrong password	Error “Invalid email or password”	
TC2	Access recruiter page as candidate	Candidate session calls /api/jobs	Request refused (401)	
TC3	Upload unsupported file	A .docx or image file	Friendly error; nothing stored	
TC4	Upload same resume twice for a job	Identical PDF	Second copy skipped as duplicate	
TC5	Batch screening	5 resumes, 1 job	5 ranked candidates with reasons	
TC6	Hallucination control	Resume lacking a skill the model reports	Unsupported item removed; count shown	
TC7	Overlapping interviews	Two bookings at the same time	Second booking rejected	
TC8	Blind screening	Switch on and open the candidate list	Names/contact details not shown	
TC9	Job application	Candidate applies with a resume	Application appears for the recruiter; status visible to candidate	
TC10	Admin monitoring	Trigger an AI error	Error visible in API monitor and audit log	

Table 10.1: Test cases

11. Conclusion

The proposed model and system design of HireSense AI have been described using a block diagram, flowcharts, algorithms, data flow diagrams and an ER diagram, and implemented as a single Next.js application with a MySQL database. The design keeps AI output explainable and verified against the source text, separates the three user roles with access control, and leaves the final hiring decision with a human. The running model, its code and the test cases are submitted for approval by the guide.

Approval by Guide

The proposed model / system design, the implementation of the code and the running model have been reviewed and approved.

Guide name	

Signature	
Date	
Remarks	

12. References

[1] Next.js Documentation, Vercel Inc. [Online]. Available: https://nextjs.org/docs

[2] MySQL 8.0 Reference Manual, Oracle Corporation. [Online]. Available: https://dev.mysql.com/doc/

[3] OpenAI API Documentation. [Online]. Available: https://platform.openai.com/docs

[4] pdf-parse package, npm registry. [Online]. Available: https://www.npmjs.com/package/pdf-parse

[5] Equal Employment Opportunity Commission et al., "Uniform Guidelines on Employee Selection Procedures," 29 C.F.R. Part 1607, 1978.

[6] Project synopsis: "HireSense AI – An AI-Based Resume Screening and Recruitment Assistant" (Synopsis.docx), including the literature references listed there.